

Un cliente de API para el resumidor

PF1822 · Especialización en Desarrollo con IA

Módulo 2 · MA04576 · INTRODUCCIÓN A MODELOS GENERATIVOS Y CONSUMO POR API · 21 h

TÉCNICA	TIEMPO ESTIMADO	INDICADORES DE LOGRO
Resolución de problemas	7 h (2 h guiadas + 5 h autónomas)	1.2, 1.3, 2.1, 2.2, 2.3, 3.1, 3.2
APOYOS	PRODUCTO	SE EVALÚA CON
Notebook guiado "Laboratorio de prompts"; Cuatro cápsulas de contenido, una por AE	Diagrama + <code>cliente_ia.py</code> + <code>test_cliente_ia.py</code> + <code>prompts.md</code>	Rúbrica de revisión de la solución (observación) (M2-Instrumento-1.pdf)

APRENDIZAJES ESPERADOS

AE1. ANALIZAR EL ROL DE LOS MODELOS GENERATIVOS EN EL DESARROLLO DE APLICACIONES, DESCRIBIENDO SUS TIPOS, CASOS DE USO, TECNOLOGÍAS ASOCIADAS Y REPRESENTANDO ARQUITECTURAS FUNCIONALES QUE INTEGREN ESTOS MODELOS MEDIANTE APIS.

AE2. UTILIZAR UN MODELO DE IA A TRAVÉS DE API REST UTILIZANDO SOLICITUDES HTTP Y AUTENTICACIÓN, BASÁNDOSE EN LA DOCUMENTACIÓN OFICIAL DE OPENAI Y HUGGING FACE.

AE3. FORMULAR PROMPTS EFECTIVOS PARA MODELOS GENERATIVOS EN ESCENARIOS ZERO-SHOT O FEW-SHOT, CONSIDERANDO LA TAREA SOLICITADA Y LOS PARÁMETROS DE GENERACIÓN DISPONIBLES.

La mesa de ayuda de Nube Sur (empresa ficticia) quiere un resumidor de tickets. Antes de escribir prompts, el equipo necesita dos cosas: saber cómo va a estar armada la aplicación y tener una pieza de código confiable para hablar con el modelo.

Parte A — Arquitectura (AE1). Dibuja el diagrama funcional del resumidor con estos componentes: la aplicación de la mesa de ayuda (cliente), el servidor que expone la API interna, el módulo de preprocesamiento, el modelo de IA externo y una base de datos o *vector store*. Explica en una línea qué hace cada uno y responde: **¿dónde se guarda la clave de la API y por qué no puede estar en el cliente?** Justifica tu arquitectura con dos criterios: escalabilidad, seguridad, costo o mantenimiento.

Parte B — Cliente de API (AE2). Programa en Python el módulo `cliente_ia.py` con una clase `ClienteIA` que:

1. lea la clave y los nombres de modelo desde **variables de entorno**, nunca desde el código;
2. liste los modelos disponibles con una solicitud **GET**;
3. genere texto con una solicitud **POST** al endpoint de chat, con `temperature` y límite de tokens configurables, y devuelva el texto y los tokens usados;
4. obtenga **embeddings** de una lista de textos con una solicitud **POST**;
5. informe los errores HTTP con un mensaje claro;

6. esté documentada con **docstrings**.

Escribe además `test_cliente_ia.py` con **al menos tres pruebas unitarias** que se ejecuten **sin conexión**, simulando la API.

Parte C — El prompt que no sirve (AE3). El equipo probó este prompt y el jefe de soporte lo rechazó:

```
Resume esto: <ticket>
```

Con `temperature` 1,0 y el ticket *"La app móvil se cierra al abrir el historial de pedidos en Android 14; en iOS funciona."* devolvió dos salidas distintas: (a) *"Hay un problema con la app. Se recomienda reinstalarla y actualizar el sistema operativo para solucionar el error de forma definitiva."* y (b) *"App falla."*

1. Diagnostica **tres fallas** del prompt o de sus parámetros: qué indicación falta y qué efecto tuvo en las salidas.
2. Reescribe el prompt como **zero-shot** con rol, tarea, formato y restricciones. Elige `temperature` y límite de tokens y justifica cada uno en una línea.
3. Pruébalo con tu `ClienteIA` sobre ese ticket y sobre este otro: *"No puedo descargar la factura de agosto desde el portal; el botón queda cargando."*

Entrega: el diagrama (imagen o Mermaid), los dos archivos `.py`, la salida de `pytest`, `prompts.md` con el diagnóstico, el prompt corregido y sus dos salidas, y un `README.md` breve. Tu clave **no** puede aparecer en ningún archivo ni captura.